

Récapitulatif de ta configuration projet

Élément	Choix
Stack	Python + Django + VSCode
Modèle	Caffe pré-entraîné via OpenCV DNN
Dataset	Adience (téléchargement guidé)
Affichage âge	Simplifié : Enfant / Ado / Adulte / Senior
Interface	Upload image → résultat
Livrables	Code + Rapport + Démo + Notebook

Architecture complète du projet

```
age_detection_project/
├── age_detection/                                # App Django principale
│   ├── migrations/
│   ├── static/
│   │   ├── age_detection/
│   │   │   ├── css/style.css
│   │   │   └── js/main.js
│   ├── templates/
│   │   ├── age_detection/
│   │   │   ├── index.html                # Page upload
│   │   │   └── result.html              # Page résultat
│   ├── models/                          # Fichiers modèles Caffe
│   │   ├── age_net.caffemodel
│   │   ├── age_deploy.prototxt
│   │   ├── face_net.caffemodel
│   │   └── face_deploy.prototxt
│   ├── views.py                        # Logique upload + inférence
│   ├── urls.py
│   ├── forms.py                        # Formulaire upload
│   └── ml_model.py                     # Code OpenCV DNN
├── media/                              # Images uploadées
├── manage.py
├── requirements.txt
└── notebook/
    └── analyse_adience.ipynb          # Notebook Jupyter
```

Étape 1 — Installation des dépendances

Dans ton terminal VSCode :

```
# Créer un environnement virtuel
python -m venv venv
venv\Scripts\activate # Windows
# ou: source venv/bin/activate # Mac/Linux

# Installer les packages
pip install django opencv-python numpy pillow jupyter matplotlib pandas
```

Étape 2 — Télécharger les modèles Caffe

Télécharge ces 4 fichiers et place-les dans `age_detection/models/` :

Modèle détection de visage (Face Net) :

- https://raw.githubusercontent.com/opencv/opencv/master/samples/dnn/face_detector/deploy.prototxt → renommer en `face_deploy.prototxt`
- https://github.com/opencv/opencv_3rdparty/raw/dnn_samples_face_detector_20170830/res10_300x300_ssd_iter_140000.caffemodel → renommer en `face_net.caffemodel`

Modèle détection d'âge (Gil Levi) :

- https://github.com/GilLevi/AgeGenderDeepLearning/raw/master/models/age_net.caffemodel
- https://github.com/GilLevi/AgeGenderDeepLearning/raw/master/age_net_definitions/deploy.prototxt → renommer en `age_deploy.prototxt`

 **Alternatif simple** : recherche sur Google "`age_net.caffemodel`" `site:github.com` — plusieurs repos hébergent ces fichiers directement.

Étape 3 — Créer le projet Django

```
django-admin startproject age_project .
python manage.py startapp age_detection
```

Dans `age_project/settings.py`, ajoute :

```
INSTALLED_APPS = [
    ...
    'age_detection',
]
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

Étape 4 — Le cœur du projet : `ml_model.py`

```
# age_detection/ml_model.py
```

```

import cv2
import numpy as np

# Tranches d'âge du modèle Caffe
AGE_BUCKETS = ['(0-2)', '(4-6)', '(8-12)', '(15-20)',
               '(25-32)', '(38-43)', '(48-53)', '(60-100)']

# Mapping vers des catégories lisibles
def get_category(age_label):
    mapping = {
        '(0-2)': 'Enfant', '(4-6)': 'Enfant', '(8-12)': 'Enfant',
        '(15-20)': 'Adolescent',
        '(25-32)': 'Adulte', '(38-43)': 'Adulte', '(48-53)': 'Adulte',
        '(60-100)': 'Senior'
    }
    return mapping.get(age_label, 'Inconnu')

def load_models():
    face_net = cv2.dnn.readNetFromCaffe(
        'age_detection/models/face_deploy.prototxt',
        'age_detection/models/face_net.caffemodel'
    )
    age_net = cv2.dnn.readNetFromCaffe(
        'age_detection/models/age_deploy.prototxt',
        'age_detection/models/age_net.caffemodel'
    )
    return face_net, age_net

def detect_age(image_path):
    """
    Détecte les visages et prédit la tranche d'âge.
    Retourne l'image annotée et les résultats.
    """
    face_net, age_net = load_models()
    image = cv2.imread(image_path)
    h, w = image.shape[:2]
    results = []

    # Détection des visages
    blob = cv2.dnn.blobFromImage(image, 1.0, (300, 300),
                                  (104.0, 177.0, 123.0))
    face_net.setInput(blob)
    detections = face_net.forward()

    for i in range(detections.shape[2]):
        confidence = detections[0, 0, i, 2]
        if confidence > 0.6: # Seuil de confiance
            # Coordonnées du visage
            box = detections[0, 0, i, 3:7] * np.array([w, h, w, h])
            (x1, y1, x2, y2) = box.astype("int")

            # Extraire le visage
            face = image[y1:y2, x1:x2]
            if face.size == 0:
                continue

            # Prédiction de l'âge
            face_blob = cv2.dnn.blobFromImage(
                face, 1.0, (227, 227),
                (78.4263377603, 87.7689143744, 114.895847746),
                swapRB=False
            )

```

```

    )
    age_net.setInput(face_blob)
    age_preds = age_net.forward()
    age_label = AGE_BUCKETS[age_preds[0].argmax()]
    category = get_category(age_label)
    confidence_pct = round(age_preds[0].max() * 100, 1)

    # Dessiner rectangle + texte
    cv2.rectangle(image, (x1, y1), (x2, y2), (0, 200, 100), 2)
    label = f"{category} {age_label} ({confidence_pct}%"
    cv2.putText(image, label, (x1, y1 - 10),
                  cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 200, 100), 2)

    results.append({
        'category': category,
        'age_range': age_label,
        'confidence': confidence_pct
    })

# Sauvegarder l'image annotée
output_path = image_path.replace('uploads/', 'results/')
cv2.imwrite(output_path, image)
return output_path, results

```

Étape 5 — `views.py`

```

# age_detection/views.py
from django.shortcuts import render
from django.core.files.storage import FileSystemStorage
from .ml_model import detect_age
import os

def index(request):
    return render(request, 'age_detection/index.html')

def detect(request):
    if request.method == 'POST' and request.FILES.get('image'):
        img = request.FILES['image']
        fs = FileSystemStorage(location='media/uploads/')
        filename = fs.save(img.name, img)

        # Créer le dossier résultats si nécessaire
        os.makedirs('media/results', exist_ok=True)

        image_path = f'media/uploads/{filename}'
        output_path, results = detect_age(image_path)

        context = {
            'original': f'/media/uploads/{filename}',
            'result_image': f'/media/results/{filename}',
            'detections': results,
            'nb_faces': len(results)
        }
        return render(request, 'age_detection/result.html', context)

    return render(request, 'age_detection/index.html',
                  {'error': 'Aucune image fournie.'})

```

Étape 6 — `urls.py`

```
# age_detection/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('detect/', views.detect, name='detect'),
]

# age_project/urls.py
from django.conf import settings
from django.conf.urls.static import static
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('age_detection.urls')),
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Tranches d'âge — Logique de catégorisation

Tranche Caffé	Catégorie affichée	Emoji
(0-2), (4-6), (8-12)	Enfant	👶
(15-20)	Adolescent	👧
(25-32), (38-43), (48-53)	Adulte	👨
(60-100)	Senior	👴

Ordre de travail recommandé

1.  **Aujourd'hui** → Installer les dépendances + créer la structure Django
 2.  **Ensuite** → Télécharger les modèles Caffé + tester `ml_model.py` seul
 3.  **Ensuite** → Connecter à Django (views + urls + templates)
 4.  **Ensuite** → Faire le Notebook Jupyter d'analyse
 5.  **Enfin** → Rédiger le rapport
-